Complete React.js Guide

From Zero to Hero

A Comprehensive Tutorial with Examples and Practice Problems

Table of Contents

1. Introduction to React.js

What is React?

React is a JavaScript library for building user interfaces, particularly single-page applications. It was created by Facebook (now Meta) and is maintained by Facebook and a community of individual developers and companies. React allows developers to create large web applications that can update and render efficiently in response to data changes.

Why Use React?

- **Component-Based Architecture:** Build encapsulated components that manage their own state, then compose them to make complex UIs
- **Declarative:** Design simple views for each state in your application, and React will efficiently update and render just the right components when your data changes
- **Virtual DOM:** React creates an in-memory data structure cache, computes the resulting differences, and then updates the browser's displayed DOM efficiently
- **Reusability:** Components can be reused throughout your application, reducing code duplication
- **Large Ecosystem:** Huge community support, thousands of third-party libraries, and extensive documentation
- **Performance:** Virtual DOM and efficient diffing algorithm make React extremely fast
- **SEO Friendly:** Can be rendered on the server using Next.js or similar frameworks
- **Mobile Development:** React Native allows you to build mobile apps using the same concepts

React vs Vanilla JavaScript

In vanilla JavaScript, you manipulate the DOM directly, which can become complex and error-prone in large applications. React provides a simpler mental model where you describe what the UI should look like, and React handles the DOM updates for you.

2. Setting Up Your Development Environment

Prerequisites

Before starting with React, you need to have the following installed:

- **Node.js (version 14 or higher):** Download and install from nodejs.org. Node.js includes npm (Node Package Manager)
- **Code Editor:** Visual Studio Code (highly recommended), Sublime Text, Atom, or any text editor you prefer
- **Basic JavaScript Knowledge:** Understanding of ES6+ features like arrow functions, destructuring, spread operator, modules, and promises
- **Command Line Basics:** Familiarity with using terminal/command prompt to run commands

Checking Your Installation

Open your terminal and run these commands to verify installation:

```
node --version
npm --version

# You should see version numbers like:
# v18.17.0
# 9.6.7
```

Essential VS Code Extensions

These extensions will significantly improve your React development experience:

- **ES7+ React/Redux/React-Native snippets:** Provides shortcuts for React code
- **Prettier - Code formatter:** Automatically formats your code
- **ESLint:** Identifies and reports patterns in JavaScript code
- **Auto Import:** Automatically imports modules
- **Bracket Pair Colorizer:** Colors matching brackets for better readability
- **Path Intellisense:** Autocompletes filenames and paths

3. Creating Your First React Project

Method 1: Create React App (Best for Beginners)

Create React App is the official tool for creating React applications with zero configuration. It sets up your development environment so you can use the latest JavaScript features, provides a nice developer experience, and optimizes your app for production.

Step-by-Step Guide:

1. Open your terminal or command prompt
2. Navigate to the folder where you want to create your project
3. Run the following command:

```
npx create-react-app my-first-app
```

This will create a new folder called 'my-first-app' with all necessary files and dependencies.

4. Navigate into your project folder:

```
cd my-first-app
```

5. Start the development server:

```
npm start
```

This command starts the development server and automatically opens your browser at http://localhost:3000. You should see the default React welcome page with the React logo spinning!

Understanding the Project Structure

Here's what the generated project structure looks like:

```
my-first-app/
■■■ node_modules/          (Dependencies - don't edit)
■■■ public/
■   ■■■ index.html        (Main HTML file)
■   ■■■ favicon.ico       (Website icon)
■   ■■■ robots.txt
■■■ src/
■   ■■■ App.js            (Main React component)
■   ■■■ App.css           (Styles for App component)
■   ■■■ App.test.js       (Test file)
■   ■■■ index.js          (Entry point)
■   ■■■ index.css         (Global styles)
■   ■■■ logo.svg          (React logo)
■   ■■■ reportWebVitals.js
■■■ .gitignore            (Git ignore file)
■■■ package.json          (Project configuration)
■■■ package-lock.json     (Dependency tree)
■■■ README.md             (Documentation)
```

Key Files Explained:

- **public/index.html:** The single HTML file in your app. React injects your components here
- **src/index.js:** The entry point of your React app. This is where React starts rendering

- **src/App.js:** Your main component. This is where you'll write most of your code
- **package.json:** Contains project metadata and dependencies
- **node_modules:** Folder containing all installed packages (created by npm)

Method 2: Vite (Faster and Modern)

Vite is a modern build tool that's significantly faster than Create React App. It's recommended for new projects if you want better performance.

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

4. Understanding React Fundamentals (JSX)

What is JSX?

JSX stands for JavaScript XML. It's a syntax extension for JavaScript that allows you to write HTML-like code in your JavaScript files. JSX makes it easier to write and understand the structure of your components. Under the hood, JSX is transformed into regular JavaScript function calls.

Basic JSX Example

```
function Welcome() {
    const name = "John";
    return (
        <div>
            <h1>Hello, {name}!</h1>
            <p>Welcome to React</p>
        </div>
    );
}
```

Important JSX Rules

- **Must return a single parent element:** You can't return multiple elements at the root level. Wrap them in a div or Fragment
- **Use className instead of class:** 'class' is a reserved keyword in JavaScript, so use 'className' for CSS classes
- **Close all tags:** Self-closing tags like img and br must have a closing slash: <img />,
- **Use camelCase for attributes:** onClick instead of onclick, backgroundColor instead of background-color
- **JavaScript expressions in curly braces:** You can embed any JavaScript expression using {curly braces}
- **Comments:** Use {/* comment */} for comments in JSX

Embedding JavaScript in JSX

```
function Greeting() {
    const user = { name: 'Alice', age: 25 };
    const isLoggedIn = true;

    return (
        <div>
            <h1>Hello, {user.name}!</h1>
            <p>You are {user.age} years old</p>
            {isLoggedIn ? <p>Welcome back!</p> : <p>Please log in</p>}
            <p>2 + 2 = {2 + 2}</p>
            <p>Random: {Math.random()}</p>
        </div>
    );
}
```

JSX Attributes

```
function ImageComponent() {
    const imageUrl = "https://example.com/image.jpg";
    const altText = "Description";

    return (
        <div>
            <img src={imageUrl} alt={altText} />
            <button className="btn-primary">Click Me</button>
            <input type="text" disabled={false} />
            <div style={{color: 'red', fontSize: '20px'}}>
                Styled Text
            </div>
        </div>
    );
}
```

React Fragments

When you need to return multiple elements without adding an extra div to the DOM, use React Fragments:

```
function List() {
    return (
        <>
            <li>Item 1</li>
            <li>Item 2</li>
            <li>Item 3</li>
        </>
    );
}

// Or with full syntax:
function List() {
    return (
        <React.Fragment>
            <li>Item 1</li>
            <li>Item 2</li>
        </React.Fragment>
    );
}
```

5. Components Deep Dive

What are Components?

Components are the building blocks of React applications. They are independent, reusable pieces of code that return React elements. Think of components as JavaScript functions that accept inputs (called props) and return React elements describing what should appear on the screen.

Types of Components

1. Functional Components (Modern Standard)

Functional components are simple JavaScript functions that return JSX. This is the recommended way to write components.

```
// Simple functional component
function Welcome(props) {
    return <h1>Hello, {props.name}</h1>;
}

// Arrow function component
const Welcome = (props) => {
    return <h1>Hello, {props.name}</h1>;
};

// With destructuring (most common)
const Welcome = ({ name }) => {
    return <h1>Hello, {name}</h1>;
};

// Implicit return (for simple components)
const Welcome = ({ name }) => <h1>Hello, {name}</h1>;
```

2. Class Components (Legacy - Not Recommended)

```
import React, { Component } from 'react';

class Welcome extends Component {
    render() {
        return <h1>Hello, {this.props.name}</h1>;
    }
}
```

Note: Modern React development uses functional components with Hooks. Class components are mainly seen in legacy codebases.

Complete Component Example

```
// UserCard.js
function UserCard({ name, role, imageUrl }) {
    return (
        <div className="user-card">
            <img src={imageUrl} alt={name} />
            <h3>{name}</h3>
            <p>{role}</p>
```

```
        </div>
    );
}

export default UserCard;
```

Using the Component

```
// App.js
import UserCard from './UserCard';

function App() {
    return (
        <div>
            <h1>Our Team</h1>
            <UserCard
                name="Alice Johnson"
                role="Frontend Developer"
                imageUrl="/alice.jpg"
            />
            <UserCard
                name="Bob Smith"
                role="Backend Developer"
                imageUrl="/bob.jpg"
            />
        </div>
    );
}

export default App;
```

Component Naming Conventions

- **Always use PascalCase:** Component names must start with a capital letter (UserCard, not userCard)
- **Be descriptive:** Use meaningful names that describe what the component does
- **One component per file:** Keep each component in its own file for better organization
- **File naming:** Match the file name to the component name (UserCard.js for UserCard component)

6. State Management with Hooks

What is State?

State is a JavaScript object that holds data that may change over the lifetime of the component. When state changes, the component re-renders to reflect those changes. State is what makes React components interactive and dynamic.

The useState Hook

The useState Hook is the most commonly used Hook in React. It allows you to add state to functional components.

Basic Syntax and Example

```
import { useState } from 'react';

function Counter() {
    // Declare state variable 'count' with initial value 0
    // setCount is the function to update the state
    const [count, setCount] = useState(0);

    return (
        <div>
            <p>You clicked {count} times</p>
            <button onClick={() => setCount(count + 1)}>
                Click me
            </button>
        </div>
    );
}
```

Understanding useState

- **First element (count):** The current state value
- **Second element (setCount):** Function to update the state
- **useState(0):** The argument is the initial state value
- **State updates trigger re-renders:** When you call setCount, React re-renders the component
- **State is isolated:** Each component instance has its own state

Multiple State Variables

```
function UserProfile() {
    const [name, setName] = useState('John');
    const [age, setAge] = useState(25);
    const [email, setEmail] = useState('john@example.com');
    const [isActive, setIsActive] = useState(true);

    return (
        <div>
            <p>Name: {name}</p>
            <p>Age: {age}</p>
            <p>Email: {email}</p>
            <p>Status: {isActive ? 'Active' : 'Inactive'}</p>
```

```
            <button onClick={() => setAge(age + 1)}>
                Birthday!
            </button>
            <button onClick={() => setIsActive(!isActive)}>
                Toggle Status
            </button>
        </div>
    );
}
```

State with Objects

```
function UserForm() {
    const [user, setUser] = useState({
        name: '',
        email: '',
        age: 0
    });

    // Update single property
    const updateName = (newName) => {
        setUser({
            ...user,          // Spread existing properties
            name: newName     // Update only name
        });
    };

    // Alternative: using function form
    const updateEmail = (newEmail) => {
        setUser(prevUser => ({
            ...prevUser,
            email: newEmail
        }));
    };

    return (
        <div>
            <input
                value={user.name}
                onChange={(e) => updateName(e.target.value)}
                placeholder="Name"
            />
            <input
                value={user.email}
                onChange={(e) => updateEmail(e.target.value)}
                placeholder="Email"
            />
        </div>
    );
}
```

State with Arrays

```
function TodoList() {
    const [todos, setTodos] = useState([]);

    // Add item
```

```
    const addTodo = (text) => {
        setTodos([...todos, { id: Date.now(), text }]);
    };

    // Remove item
    const removeTodo = (id) => {
        setTodos(todos.filter(todo => todo.id !== id));
    };

    // Update item
    const updateTodo = (id, newText) => {
        setTodos(todos.map(todo =>
            todo.id === id ? { ...todo, text: newText } : todo
        ));
    };

    return (
        <div>
            {todos.map(todo => (
                <div key={todo.id}>
                    <span>{todo.text}</span>
                    <button onClick={() => removeTodo(todo.id)}>
                        Delete
                    </button>
                </div>
            ))}
        </div>
    );
}
```

State Update Patterns

- **Direct update:** setCount(5) - sets state to a specific value
- **Functional update:** setCount(prev => prev + 1) - uses previous value (safer for async updates)
- **Never mutate state directly:** Always create new objects/arrays
- **Batch updates:** React may batch multiple setState calls for performance

7. Props and Data Flow

What are Props?

Props (short for properties) are how you pass data from parent components to child components in React. Props are read-only (immutable) - a child component cannot modify its props. Think of props as function arguments.

Basic Props Example

```
// Child Component
function Greeting({ name, age }) {
    return (
        <div>
            <h1>Hello, {name}!</h1>
            <p>You are {age} years old</p>
        </div>
    );
}

// Parent Component
function App() {
    return (
        <div>
            <Greeting name="Alice" age={25} />
            <Greeting name="Bob" age={30} />
        </div>
    );
}
```

Props Types

```
function Example() {
    // String props (quotes)
    <Component text="Hello" />

    // Number props (curly braces)
    <Component count={42} />

    // Boolean props
    <Component isActive={true} />

    // Array props
    <Component items={['a', 'b', 'c']} />

    // Object props
    <Component user={{ name: 'Alice', age: 25 }} />

    // Function props
    <Component onClick={handleClick} />
}
```

Default Props

```
function Button({ text = 'Click Me', color = 'blue' }) {
```

```
    return (
        <button style={{ backgroundColor: color }}>
            {text}
        </button>
    );
}

// Usage
<Button />  // Uses all defaults
<Button text="Submit" />  // Custom text, default color
<Button text="Delete" color="red" />  // Custom both
```

Children Prop

The children prop is special - it represents the content between opening and closing tags of a component.

```
function Card({ title, children }) {
    return (
        <div className="card">
            <h2>{title}</h2>
            <div className="card-body">
                {children}
            </div>
        </div>
    );
}

// Usage
<Card title="My Card">
    <p>This is the content</p>
    <button>Click Me</button>
    <img src="image.jpg" />
</Card>
```

Passing Functions as Props

```
function Parent() {
    const handleClick = (message) => {
        alert(message);
    };

    return <Child onButtonClick={handleClick} />;
}

function Child({ onButtonClick }) {
    return (
        <button onClick={() => onButtonClick('Hello from child!')}>
            Click Me
        </button>
    );
}
```

8. Event Handling

Handling Events in React

React events are named using camelCase (onClick, onChange) and you pass a function as the event handler, not a string.

Common Events

```
function EventsDemo() {
    // Click handler
    const handleClick = () => {
        console.log('Button clicked!');
    };

    // Input change handler
    const handleChange = (event) => {
        console.log('Input value:', event.target.value);
    };

    // Form submit handler
    const handleSubmit = (event) => {
        event.preventDefault();
        console.log('Form submitted!');
    };

    // Mouse events
    const handleMouseEnter = () => {
        console.log('Mouse entered!');
    };

    const handleMouseLeave = () => {
        console.log('Mouse left!');
    };

    return (
        <div>
            <button onClick={handleClick}>Click Me</button>

            <input onChange={handleChange} />

            <form onSubmit={handleSubmit}>
                <button type="submit">Submit</button>
            </form>

            <div
                onMouseEnter={handleMouseEnter}
                onMouseLeave={handleMouseLeave}
            >
                Hover over me
            </div>
        </div>
    );
}
```

Event Object

```
function InputExample() {
    const handleChange = (event) => {
        console.log('Value:', event.target.value);
        console.log('Name:', event.target.name);
        console.log('Type:', event.type);
    };

    const handleKeyPress = (event) => {
        if (event.key === 'Enter') {
            console.log('Enter key pressed!');
        }
    };

    return (
        <input
            name="username"
            onChange={handleChange}
            onKeyPress={handleKeyPress}
        />
    );
}
```

Passing Arguments to Event Handlers

```
function ItemList() {
    const handleDelete = (id) => {
        console.log('Deleting item:', id);
    };

    const items = [1, 2, 3, 4, 5];

    return (
        <div>
            {items.map(item => (
                <div key={item}>
                    <span>Item {item}</span>
                    {/* Method 1: Arrow function */}
                    <button onClick={() => handleDelete(item)}>
                        Delete
                    </button>
                </div>
            ))}
        </div>
    );
}
```

9. Lists and Keys

Rendering Lists

In React, you transform arrays into lists of elements using the JavaScript map() function.

```
function NumberList() {
    const numbers = [1, 2, 3, 4, 5];

    return (
        <ul>
            {numbers.map((number) => (
                <li key={number}>{number}</li>
            ))}
        </ul>
    );
}
```

Why Keys Matter

Keys help React identify which items have changed, been added, or removed. They give elements a stable identity and help React optimize rendering.

- **Keys must be unique among siblings**
- **Don't use array index as key if order may change**
- **Use stable IDs from your data when possible**
- **Keys don't get passed to components as props**

Complex List Example

```
function UserList() {
    const users = [
        { id: 1, name: 'Alice', email: 'alice@example.com' },
        { id: 2, name: 'Bob', email: 'bob@example.com' },
        { id: 3, name: 'Charlie', email: 'charlie@example.com' }
    ];

    return (
        <div>
            {users.map((user) => (
                <div key={user.id} className="user-card">
                    <h3>{user.name}</h3>
                    <p>{user.email}</p>
                </div>
            ))}
        </div>
    );
}
```

Quick Reference Guide

Essential React Concepts Summary

- **Components:** Building blocks of React apps - reusable pieces of UI
- **JSX:** Syntax that looks like HTML but works in JavaScript
- **Props:** Data passed from parent to child (read-only)
- **State:** Data that changes over time (managed with useState)
- **Events:** Handle user interactions (onClick, onChange, etc.)
- **Lists:** Render arrays using map() with unique keys
- **Hooks:** Functions that let you use React features (useState, useEffect, etc.)

Common Commands

```
# Create new React app
npx create-react-app my-app

# Start development server
npm start

# Build for production
npm run build

# Install a package
npm install package-name

# Install React Router
npm install react-router-dom

# Install Axios for HTTP requests
npm install axios
```

Next Steps in Your React Journey

- Build small projects (todo app, weather app, quiz app)
- Learn useEffect for side effects and data fetching
- Master forms and form validation
- Explore React Router for multi-page apps
- Learn about Context API for global state
- Study popular state management libraries (Redux, Zustand)
- Explore Next.js for server-side rendering
- Learn TypeScript with React
- Practice testing with Jest and React Testing Library

Important Resources

- **Official Docs:** react.dev (best resource!)
- **React Tutorial:** react.dev/learn
- **Community:** Reactiflux Discord, Reddit r/reactjs
- **Practice:** freeCodeCamp, Scrimba, Frontend Mentor
- **Advanced Learning:** Epic React by Kent C. Dodds

Congratulations! You now have a solid foundation in React.js. The key to mastering React is consistent practice - build projects, experiment with code, make mistakes, and learn from them. Happy coding!